

Multi-Source Candidate Data Transformer — Technical Design

Eightfold Engineering Intern Assignment | Step 1 Deliverable

1. Problem & Approach

Candidate data arrives from structured sources (ATS JSON, recruiter CSV) and unstructured sources (resume PDF, recruiter notes) that disagree on field names and sometimes on facts. The pipeline turns this into one clean, deduplicated, schema-valid, provenance-tracked candidate profile, with the output shape controlled entirely by a runtime config (no code changes).

2. Pipeline Architecture

Detect → **Extract** → **Parse** → **Normalize** → **Merge** → **Resolve** → **Validate** → **Project**. Each stage is one focused module so a new source type or normalization rule only touches one file:

- **extractor.py (Detect+Extract)** — maps file extension → reader fn; returns raw dict/rows/text. Unknown extensions and unreadable files are skipped with a warning, never crash the run.
- **parser.py (Extract→Output, field-name resolution)** — structured sources are mapped via per-field alias lists (e.g. full_name: name[full_name|candidate_name|...] since every exporter spells fields differently. Unstructured sources are parsed with regex heuristics (Name:/Skills: labels, email/phone/LinkedIn/GitHub patterns, an 'N years experience' phrase).
- **normalizer.py (Normalize)** — pure, independently-testable functions: email → lowercase/trim; phone → E.164 via the phonenumbers library (default region IN, digit-only fallback if unparseable); country → ISO-3166 alpha-2 via pycountry fuzzy match; dates → YYYY-MM; skills → canonical display name via a small alias table (js→JavaScript, etc.).
- **merger.py (Merge + Conflict-resolution)** — groups parsed records into one candidate per real person, then resolves each field (see Sections 4-5).
- **validator.py (Validate)** — forces every output record into the exact schema shape; any missing or wrong-typed field becomes explicit null instead of an exception or a missing key.
- **config_loader.py (Project)** — re-shapes the validated internal record into the user-configured output document (rename/select/nest/omit), per Section 6.

3. Extraction & Normalization

Extraction is deliberately 'dumb' (no renaming/cleaning) so adding a new file type is a one-function change. Normalization functions are pure (value in, clean value or None out) which makes them trivially unit-testable and reusable regardless of which source the value came from.

4. Merge / Conflict-Resolution Policy

Rule	Logic
Identity (merge)	Same person if normalized emails overlap (strong signal); else if no email overlap, fuzzy name match (rapidfuzz token_sort_ratio ≥ 88) and no conflicting email. Prevents false merges of different people who share a common name.
Conflict (scalars)	Configurable SOURCE PRIORITY list, default resume_pdf > ats_json > recruiter_csv > recruiter_notes — a candidate's own resume is closest to ground truth; ATS intake is usually careful; CSV exports can be stale; free-text notes are least structured. Highest-priority non-null value wins; every decision is logged to provenance: {[field, source, method]}.
Lists (emails/phones/skills)	UNIONed, never conflict-resolved — more contact info / corroborated skills is strictly more useful. A skill seen in N sources gets a small confidence boost (capped at 0.99).
Experience/education	Merged by (company,title) / (institution,degree) key; on a key collision the higher-priority source's entry wins, consistent with the scalar rule above.

5. Runtime Configuration (no code changes)

config.json declares output 'fields' as {[path, from, type, required, normalize]}. 'from' resolves a dotted path (with [] for list iteration) against the internal canonical record; 'path' is where it's written in the output, so fields can be renamed and re-nested freely. 'on_missing' controls null vs. omit vs. error behavior per-config. The internal engine (extract/normalize/merge) never changes — only this final reshape does — satisfying 'changing the config changes the output, not the code.' Two example configs are included: the default schema and a renamed/nested/omit-on-missing variant.

6. Validation & Robustness

Every stage degrades instead of crashing: unsupported file extensions are skipped with a warning; unparseable/garbage PDFs yield empty text rather than raising; malformed JSON/CSV rows are caught and logged; validator.py coerces any wrong-typed or missing field to null against a fixed schema; a top-level try/except in main.py guarantees the CLI always writes a valid (possibly empty) JSON array. Verified against an empty input folder, a syntactically broken JSON file, an unrecognized extension, and a fully blank candidate record.

7. Assumptions & Edge Cases

- Default phone region for E.164 parsing is IN (sample data is India-based); a digit-only fallback is used if a number can't be parsed.
- Free-text parsing uses lightweight regex heuristics rather than an LLM/NLP pipeline, to stay dependency-light and deterministic for grading — documented as a lower-priority trade-off per the brief.
- Two people sharing a name but no email never merge (identity rule requires email overlap OR no-conflicting-email + name match).
- A scalar conflict always defers to source priority rather than recency/length heuristics, to keep the rule simple, deterministic, and auditable.
- Completely empty/garbage source rows still produce a schema-valid all-null candidate record rather than being silently dropped, keeping data-quality issues visible.